AIQA — Phase 6 Run-Level ERROR Handling Fix Report

Date: 2026-08-25 **Branch:** `claude/run-this-9eeb2e` **Scope:** Make `PerfRunState::Error`/`PerfVerdict::Error` genuinely, reliably reachable when the background performance workload task fails unexpectedly — without redesigning the workload engine, without rewriting Load/Stress/Spike/Volume logic, without introducing any fake or frontend-only error state.

---

1. Executive Summary

`ERROR` was a defined enum variant with no real path to it. The workload task was spawned via a bare, unobserved `tokio::spawn(...)`; if that task ever panicked, the entire future — including the finalization call sequenced after it — simply vanished. The persisted row stayed at `RUNNING` until the *next process restart's* recovery sweep eventually, indirectly, noticed and marked it `INTERRUPTED`. This was disclosed as a known limitation in the original Phase 6 report and is exactly what this fix closes.

The fix adds real Tokio task supervision: the workload is still spawned exactly as before, but its `JoinHandle` is now owned and awaited by a supervisor that reacts to `Ok(())` (normal completion — unchanged behavior) versus `Err(JoinError)` (a genuine, unexpected task failure — a real panic or abort), and finalizes the latter as a real, immediately-persisted `ERROR` with a safe reason, real partial metrics preserved from whatever samples had already completed, and a frozen `finished_at`. A real DB-level idempotency guard was also added so a terminal row can never be overwritten by a later finalize attempt.

Live-verified: a real normal Load run still completes normally; a real cancellation still resolves as `CANCELLED`, never `ERROR`; a real process crash mid-run still recovers as `INTERRUPTED`, distinctly from `ERROR`. A genuine, deterministic Tokio panic reaching persisted `ERROR` with a real, safe reason and real partial metrics is proven by 5 new integration tests exercising the actual production code path end-to-end (not mocked) — the one thing this fix could not do is trigger that same panic through the real, running HTTP server without modifying the untouched Load/Stress/Spike/Volume logic, which is honestly disclosed in §19 rather than faked.

Final Verdict: FIXED. See §24.

---

2. Original Issue

`PerfRunState::Error` and `PerfVerdict::Error` existed in the type system from the original Phase 6 implementation, correctly documented as "reserved for a genuine future infra-fault path... not currently reachable." That disclosure was accurate: nothing in the codebase ever constructed a `PerfRunState::Error`. The only two states `run_workload` itself ever set were `Completed`/`Cancelled`. The only way a run could end up looking like something-went-wrong was the separate, distinct `INTERRUPTED` status — assigned only by the startup recovery sweep, only after a full process restart, only for a row still `RUNNING` at that point. A workload task panicking while the process kept running (the one class of failure this system can genuinely have, since `run_workload` has no fallible internal step by design — see §7) had **no real, immediate detection path at all**.

3. Fresh Baseline Audit

Performed before any change, per the brief's own required ordering:

- `git status` — clean working tree, matching the state left by the prior Phase 6 session (§4).
- Located `PerfVerdict`/`PerfRunState` (`performance_testing.rs`) — both already had `Error` variants, both confirmed unreachable by reading `run_workload`'s own terminal-state-setting line (`let final_state = if ... Cancelled else Completed`) and `evaluate_verdict` (no `Error`-producing branch).
- Located the spawn point: `api.rs::start_performance_test_scoped`, a bare `tokio::spawn(async move { run_workload(...).await; finalize_performance_run(...).await; })` — `JoinHandle` never bound to a variable, never awaited, never observed.
- Located `finalize_performance_run` — the only finalization path, written unconditionally after `run_workload` returns; if `run_workload` panics, this line is never reached.
- Located `error_reason: Arc<RwLock<Option<String>>>` on `PerformanceRunHandle` — defined, but never written to and never read from anywhere.
- Located `recover_incomplete_performance_runs`/`list_incomplete_performance_runs` (`main.rs`/`store.rs`) — the real, working startup-recovery-to-`INTERRUPTED` path, confirmed unaffected in scope by this fix (still governs only "still `RUNNING` at process start," a genuinely different trigger from "task panicked while the process is still up").
- Located `get_performance_run_status_scoped` — confirmed it would, before this fix, incorrectly route a (hypothetical) `PerfRunState::Error` through `evaluate_verdict` (threshold comparison logic with no real ERROR path of its own) rather than reporting the real reason.
- Located `Store::finalize_performance_run` — confirmed its `UPDATE ... WHERE id = ?` had no status guard, meaning a second finalize call for the same run would silently overwrite an already-terminal row.
- Located the frontend's existing `PerformanceRunLifecycleStatus`/`getStatusBadge` — confirmed `'ERROR'` was already a valid type-level status and already had its own badge entry (label "Error", distinct icon), but no dedicated panel rendered the actual reason text, and no live/durable-round-trip ever populated one to render in the first place.

Answering the brief's own required baseline questions directly, with code evidence:

A. Normal completion? `run_workload` returns `()`; the (bare, unobserved-until-this-fix) spawn's own next line called `finalize_performance_run`, which read `handle.samples`/`handle.state` and persisted `COMPLETED`/`PASSED`/`FAILED`/`CANCELLED` via `evaluate_verdict`. Unchanged by this fix.

B. Cancellation? `run_workload` itself checks `handle.user_cancelled` at its own end and sets `PerfRunState::Cancelled` before returning `()` — cancellation was always a *normal* return, never a panic. Unchanged by this fix; still resolves through the supervisor's `Ok(())` branch (§8, §13).

C. An individual HTTP request fails? `execute_plan_once` classifies it as `SampleOutcome::Error` (a real per-request outcome), pushed into `handle.samples`; `run_workload` continues normally. This was always correctly *not* a run-level failure and remains so — verified by both a pre-existing unit test and a new regression test (§17.C).

D. An internal worker returns an "unexpected error"? There is no such return type in `run_workload`'s design — every internal step is infallible by construction (see §7). The only genuinely-possible "internal worker failure" is a Rust panic.

E. The top-level spawned task panics? Before this fix: the entire spawned future aborts; `finalize_performance_run` (sequenced after the panicking call, in the same async block) never runs; the row is silently stuck at `RUNNING`. This is the exact gap fixed.

F. Was the `JoinHandle` observed? No — `tokio::spawn(...)` was called for its side effect only, its return value discarded.

G. Can a run remain `RUNNING` after its background task exits (via panic)? Yes, before this fix — durably, until the next restart. Fixed (§8).

H. When does `INTERRUPTED` get assigned? Only by `recover_incomplete_performance_runs`, only at process startup, only for a row the DB shows as `RUNNING` at that moment. Unchanged, still exactly this — confirmed still correctly distinct from the new `ERROR` path by live verification (§20, Test 4).

I. Was `ERROR` reachable from a genuine run-level failure? No. Now: yes (§8).

4. Files Inspected

`performance_testing.rs`, `api.rs`, `store.rs`, `state.rs`, `main.rs`, `api_tests.rs`, `PerformanceTestingPage.tsx`, `PerformanceTestingPage.test.tsx`, `contract.ts` — every file the brief's own audit checklist named, all read in full or in the relevant part before any edit.

5. Files Changed

File	Change	Why
`services/orchestrator/src/performance_testing.rs`	Added `SupervisionFailure` struct + `supervise_workload_task<F>` function; added 5 new unit/integration-style tests	The real supervision mechanism itself — owns the `JoinHandle`, converts a real `JoinError` into a safe, sanitized `SupervisionFailure`
`services/orchestrator/src/api.rs`	`start_performance_test_scoped`'s spawn now wraps the workload in `supervise_workload_task` and matches on the result; added `finalize_performance_run_as_error` (`pub(crate)`); `get_performance_run_status_scoped` now special-cases `PerfRunState::Error` (real reason, not routed through `evaluate_verdict`); `PerformanceRunStatusResponse` gained an explicit `error_reason` field	The real wiring — detection, immediate finalization, correct live/durable status reporting
`services/orchestrator/src/store.rs`	`finalize_performance_run`'s `UPDATE` gained `AND status = 'RUNNING'`	Real, DB-level terminal-state idempotency guard (§13)
`services/orchestrator/src/api_tests.rs`	5 new integration tests (`phase6_error_*`)	Real, deterministic, HTTP-round-trip proof of the fixed lifecycle (§17)
`forentend/src/types/contract.ts`	`PerformanceRunStatusResponse` gained `error_reason: string \	null`
`forentend/src/components/pages/PerformanceTestingPage.tsx`	New, dedicated "Execution Error" panel, shown only for `status === 'ERROR'`, rendering the real `error_reason`; Verdict Reasons panel now explicitly excludes the `ERROR` case (no duplicate rendering)	Distinct, truthful UI for a genuine run failure — never merged into the generic reasons list, never shown as FAIL
`forentend/src/components/pages/PerformanceTestingPage.test.tsx`	1 new test	Proves the real UI distinction (§17.K)

No file outside this list was touched. `run_workload`, `run_load`, `run_stress`, `run_spike`, `run_volume`, `spawn_worker`, `compute_metrics`, `evaluate_verdict`, `execute_plan_once` — every piece of working Load/Stress/Spike/Volume logic — are **byte-for-byte unchanged** by this fix (verified: `git diff` shows no hunks inside those functions).

6. Previous Lifecycle

```
POST /start
    v
insert_performance_run (status=RUNNING)
    v
tokio::spawn(async move {
    run_workload(...).await;        <-- if this panics, everything below
                                        never runs, and the JoinHandle
                                        that could have told anyone is
                                        discarded
    finalize_performance_run(...).await;
})
    v
[normal case] finalize_performance_run -> COMPLETED/PASSED/FAILED/CANCELLED
[panic case]  <nothing> -> row stuck at RUNNING -> next restart -> INTERRUPTED
```

7. Root Cause

Exactly: a detached `tokio::spawn` with an ignored `JoinHandle` (brief's own hypothesis, confirmed by direct code inspection, not guessed). Not a race, not missing `Result` handling inside `run_workload` (it has none to miss — every step is infallible by design; there is no `Result`-returning internal call whose `Err` arm was being silently dropped). Not a missing finalization *path* so much as a finalization *call sequenced after a line that can vanish the entire task if it panics*.

8. Final Lifecycle

```
POST /start
    v
insert_performance_run (status=RUNNING)
    v
tokio::spawn(async move {
    let outcome = supervise_workload_task(async move {
        run_workload(...).await;   <-- byte-for-byte unchanged
    }).await;
    match outcome {
        Ok(())        => finalize_performance_run(...).await,          <-- unchanged path
        Err(failure)  => finalize_performance_run_as_error(...).await, <-- NEW
    }
})
```

- **Normal completion:** `run_workload` returns `()` normally (having internally set `Completed`) -> `supervise_workload_task` returns `Ok(())` -> `finalize_performance_run`, unchanged.
- **Cancellation:** `run_workload` returns `()` normally (having internally set `Cancelled`) -> `Ok(())` -> `finalize_performance_run`, unchanged. Live-verified §20 Test 3.
- **Threshold failure:** a normal completion whose `evaluate_verdict` call finds a violated threshold -> `Ok(())` -> `finalize_performance_run` -> `FAILED`. Unchanged, still distinct from `ERROR` (§9, §17.B/§17.G).
- **Request-level failure:** recorded as a real `SampleOutcome::Error` sample inside `run_workload`; the run itself still returns normally -> `Ok(())`. Unchanged, still not a run-level `ERROR` (§9, §17.C).
- **Internal execution error / task panic:** the spawned `run_workload` future panics -> `tokio::spawn`'s own real `JoinHandle` resolves `Err(JoinError)` -> `supervise_workload_task` converts it to `Err(SupervisionFailure)` -> `finalize_performance_run_as_error` -> real, immediate `ERROR`, real partial metrics, real safe reason, `finished_at` frozen now. **NEW, this fix.**
- **Startup recovery:** unchanged — a row the DB shows as `RUNNING` at process start is marked `INTERRUPTED` by `recover_incomplete_performance_runs`, a real, distinct status from `ERROR`. Live-verified §20 Test 4.

9. ERROR vs Other States

State	Meaning	Trigger
`COMPLETED`	Execution finished normally; no threshold configured, so no pass/fail judgement	`run_workload` returns normally, `evaluate_verdict` finds no configured thresholds
`PASSED`	Execution finished normally; every configured threshold was met	`run_workload` returns normally, `evaluate_verdict` finds all thresholds met
`FAILED`	Execution finished normally; at least one configured threshold was violated	`run_workload` returns normally, `evaluate_verdict` finds a violation
`CANCELLED`	Execution was genuinely, explicitly cancelled	`PerformanceRunHandle::request_cancel`, observed internally by `run_workload`'s own normal return path
`ERROR` **(this fix)**	The workload *task itself* failed unexpectedly and could not complete normally — not a request-level or threshold-level outcome	`supervise_workload_task` observes a real `JoinError` (panic) from the workload's own `JoinHandle`
`INTERRUPTED`	A row was discovered still `RUNNING` at process startup — no live task to ask at all	`recover_incomplete_performance_runs`, startup only

`ERROR` and `INTERRUPTED` remain semantically and mechanically distinct: `ERROR` is detected *while the process is still up*, by a real supervisor watching a real, specific task's `JoinHandle`; `INTERRUPTED` is detected *after a restart*, by a sweep that has no live task to observe at all (the whole process that would have supervised it is gone). Live-verified as genuinely different outcomes for genuinely different failure classes (§20, Tests 2 vs 4).

10. Tokio Supervision Design

```
pub async fn supervise_workload_task<F>(workload: F) -> Result<(), SupervisionFailure>
where
    F: std::future::Future<Output = ()> + Send + 'static,
{
    match tokio::spawn(workload).await {
        Ok(())         => Ok(()),
        Err(join_err)  => Err(SupervisionFailure::from_join_error(join_err)),
    }
}
```

This is option B from the brief's own menu: "spawning a supervisor task that awaits the workload `JoinHandle`." The outer `tokio::spawn` in `start_performance_test_scoped` is now itself the supervisor; it spawns the *real* `run_workload` call as an inner task and awaits that inner `JoinHandle` via `supervise_workload_task`. `run_workload`'s own signature (`-> ()`, no `Result`) is untouched — the "nested Result" case the brief anticipated (`Ok(Ok(...))`/`Ok(Err(...))`/`Err(join_error)`) collapses to the two real cases this codebase's actual `run_workload` can produce: `Ok(())` (it has no fallible return to wrap) or `Err(JoinError)` (the task didn't return at all). No second execution lifecycle was invented — there is exactly one `run_workload` call, wrapped, not replaced.

11. Panic Handling

```
fn from_join_error(e: tokio::task::JoinError) -> Self {
    if e.is_panic() {
        let payload = e.into_panic();
        let detail = payload.downcast_ref::<&str>().map(|s| s.to_string())
            .or_else(|| payload.downcast_ref::<String>().cloned());
        let safe_message = match detail {
            Some(s) => format!("performance execution terminated unexpectedly: {s}"),
            None => "performance execution terminated unexpectedly (internal panic)".to_string(),
        };
        Self { safe_message }
    } else {
        Self { safe_message: "performance execution task was terminated before it could finish".to_string() }
    }
}
```

Only the panic's own `&str`/`String` payload (the overwhelmingly common case for `panic!("...")`/`.unwrap()`/`.expect("...")`) is ever surfaced — never the raw `Box<dyn Any>`, never a debug dump, never a backtrace. An unformattable/non-string payload (proven live by a dedicated test using `std::panic::panic_any(42u32)`) produces a fixed, generic, safe fallback string instead of failing to format or leaking an arbitrary value. `JoinError::is_cancelled()` (task aborted) is handled as a genuinely distinct, honest branch, even though nothing in this codebase's real code path currently calls `.abort()`. `tokio::spawn` itself is what prevents a panic in the workload task from crashing the whole process — this is standard, built-in Tokio task isolation (each spawned task panicking only aborts that task), not a custom `catch_unwind` this fix had to build. Proven by 3 dedicated unit tests (§17.E) that trigger a genuine `panic!`/`panic_any` inside a real spawned task and confirm the process (and the test run) survives.

12. Partial Metrics

`finalize_performance_run_as_error` reads `handle.samples` — whatever real `RequestSample`s the workers had already pushed before the panic — and computes real metrics from exactly that set via the same, unmodified `compute_metrics` every other terminal path uses. Nothing is padded to a "planned" total; nothing is discarded. Live-proven by a real integration test (§17.D/§17.F) that pushes exactly 40 real samples, forces a real panic, and asserts the persisted/returned `metrics.total_requests`/`sample_count` are exactly `40` — never `0`, never a planned-but-never-run total.

13. Terminal-State Idempotency

- `finalize_performance_run` and `finalize_performance_run_as_error` are mutually exclusive branches of one `match` fed by one `JoinHandle` await — by construction, only one of them can ever run for a given spawned task.
- Defensively, in case any future code path ever called `Store::finalize_performance_run` a second time for the same run, the SQL itself is now guarded: `UPDATE performance_runs SET ... WHERE id = ? AND status = 'RUNNING'`. A second call against an already-terminal row is a real, honest no-op (`rows_affected() == 0`), never a silent overwrite. Proven directly by `phase6_error_cannot_later_be_overwritten_back_to_completed_terminal_idempotency` (§17.H): finalize as `ERROR`, then attempt to finalize the same run as `COMPLETED`, assert the second call reports zero rows changed and the row is still genuinely `ERROR`.
- `finished_at`/`finished_elapsed_ms` are set exactly once per run, at the moment of real detection (either `run_workload`'s own normal-completion line, or `finalize_performance_run_as_error`'s real `now - started_at` read, taken only after the task is *provably* no longer running).
- Cancellation-vs-ERROR precedence: documented explicitly. A genuine task panic always results in `ERROR`, regardless of whether a cancel was also concurrently requested — a panic means the task did not honor the cancellation gracefully, and reporting a clean `CANCELLED` in that case would hide a real failure. In practice this precedence is never actually exercised by this system's real code (cancellation and normal completion both flow through the *same* `Ok(())` branch; only a genuine crash reaches `Err`), so there is no real race to lose — documented here for completeness per the brief's own explicit request.
- Existing cancellation and startup recovery behavior: unchanged, re-verified by regression tests and live (§17, §20).

14. Persistence

PERSISTED (in the existing `performance_runs` row, no schema/migration change needed — every column this fix needs already existed):

- `status = 'ERROR'`, `verdict = 'ERROR'`
- `error_reason` — the real, safe, sanitized panic message
- `verdict_reasons_json` — now also mirrors the same reason (a real gap this fix's own tests caught: an earlier version of this fix left this `None`, so a *durable, post-restart* re-read of an `ERROR` run showed an empty reasons list even though `error_reason` was separately persisted — fixed, see §17.D vs §17.F for the before/after)
- `metrics_json` — the real, partial metrics computed from whatever samples had actually completed
- `finished_at` — frozen at real detection time

SESSION ONLY: the live `PerformanceRunHandle.error_reason`/`.state` while the process that ran it is still up — mirrors the exact same, already-disclosed limitation every other live field on this handle has.

NOT AVAILABLE: nothing new. No raw stack trace, no full panic backtrace, and no non-string panic payload are ever persisted or returned — deliberately, per §5/§11 of the fix brief ("do not expose sensitive internal secrets... safe truthful error message").

15. API Contract

`PerformanceRunStatusResponse` gained exactly one new field, additive and non-breaking: `error_reason: string | null`. Populated in both the live-handle branch (from `handle.error_reason`, only when `status === 'ERROR'`) and the durable-row fallback branch (from the persisted `row.error_reason` column directly, always — `null` for every non-error status either way). No existing field was removed, renamed, or had its meaning changed. `verdict_reasons` continues to carry the same message too, so any caller relying on the pre-existing array-based convention keeps working unmodified.

16. Frontend Handling

- `getStatusBadge` already had a distinct `ERROR` entry (different label — "Error," never "Failed" — and a different icon, `ShieldAlert` vs the `FAILED` case's `Clock`); unchanged, still correct.
- New: a dedicated `#perf-execution-error` panel, rendered only when `status === 'ERROR'`, showing the real `error_reason` (or an honest generic fallback if, somehow, none was returned) plus explanatory text distinguishing it from a threshold FAIL.
- The generic Verdict Reasons panel now explicitly excludes the `ERROR` case (`live.status !== 'RUNNING' && live.status !== 'ERROR' && ...`), so the real reason is never rendered twice under two different headings.
- No retry functionality exists on this page at all (none existed before this fix, none was added) — starting a new run always genuinely calls `POST .../start` again, there is no "reset status" affordance of any kind.

17. Tests Added

`performance_testing.rs` **(5 new):**

- A. `a_normal_workload_task_supervised_returns_ok` — a trivially-returning future supervises to `Ok`.
- E. `a_genuine_deterministic_panic_in_the_supervised_task_is_caught_not_left_to_crash_the_process` — a real `panic!` inside a real spawned task; asserts the safe message contains the real panic text, the process/test survives.
- E (payload variant). `a_panic_with_a_non_string_payload_still_produces_a_safe_generic_message` — `std::panic::panic_any(42u32)`; asserts a fixed, safe fallback string, no formatting failure.
- Regression. `a_real_normal_workload_wrapped_in_supervision_still_completes_normally` — the real, unmodified `run_workload` against a real local HTTP target, wrapped in the new supervisor, asserts `Ok(())` and `Completed`.
- G. `a_real_cancelled_workload_wrapped_in_supervision_still_reports_cancelled_not_error` — a real, genuine cancellation mid-flight through the new supervisor, asserts `Ok(())` and `Cancelled` (not `Error`) — the documented precedence rule proven in the one direction this system can actually exercise.

`api_tests.rs` **(5 new,** `phase6_error_*`**):**

- D. `phase6_error_a_genuine_internal_failure_is_detected_persisted_and_visible_live` — real project/key, real `insert_performance_run`, 40 real partial samples, live handle registered, real `supervise_workload_task` panic, real `finalize_performance_run_as_error`, real HTTP `GET .../status` — asserts `ERROR`/real reason/`finished_at`/exactly 40 partial samples/no orphan `RUNNING`.
- J. `phase6_error_persists_and_reloads_correctly_once_the_live_handle_is_gone` — same real lifecycle, but the handle is deliberately *not* registered live, forcing the durable-fallback read path; asserts the same real result on reload, and that a second read returns byte-identical metrics/`finished_at` (no drift).
- H. `phase6_error_cannot_later_be_overwritten_back_to_completed_terminal_idempotency` — proves the DB-level guard (§13).
- `phase6_error_is_distinct_from_threshold_fail_and_from_cancelled` — pure string-distinctness proof against the real `PerfVerdict`/`PerfRunState` the API actually returns.
- A. `phase6_error_normal_completion_regression_still_reaches_completed_not_error` — a real, full `POST /start` -> poll -> `GET /status` round trip through the real router, asserting the new supervision wrapper never turns a normal run into a false `ERROR`.

Frontend (1 new): `PerformanceTestingPage — genuine run-level ERROR display` — renders the real page with a mocked `ERROR` status response, asserts the label is "Error" (never "Failed"/"Passed"/"Completed"), the dedicated Execution Error panel is present with the real reason text, the generic Verdict Reasons panel is *not* also rendered, and the real partial sample count is shown.

18. Test Results

- `performance_testing.rs`: **30/30 passed** (25 pre-existing + 5 new), 0 failed.
- `api_tests.rs` new tests: **5/5 passed**, 0 failed.
- Full backend suite, no live server on port 8080: **295 passed, 0 failed** attributable to this fix (3 pre-existing, environment-dependent failures — see §19 — plus 2 ignored, Ollama-dependent, unrelated).
- Full backend suite, with a live server on port 8080: **298 passed, 0 failed, 2 ignored** — every test green, including the 3 environment-dependent ones.
- One transient, non-reproducing flake (`actual_be9_scoped_approve_and_reject_through_router`, an unrelated run-approval timing test, nothing to do with performance testing) was observed once under full-suite CPU contention, passed cleanly in isolation (0.08s) and on an immediate full-suite re-run — consistent with the same class of CPU-contention flakiness already documented in an earlier phase's own report, not a regression from this fix.
- Frontend: **401/401 passed** (400 pre-existing + 1 new), 22 test files, 0 failed.
- `npx tsc --noEmit`: clean.
- `npm run build`: succeeded, exit code 0.

19. Live Verification

Performed against a locally-rebuilt-and-restarted orchestrator, targeting only the orchestrator's own `127.0.0.1:8080/health` endpoint — no external, unauthorized load generation.

- **Test 1 — Normal Load:** a real `LOAD` test (concurrency 3, 2s) — `status: COMPLETED`, `verdict: COMPLETED`, `error_reason: null`, 9042 real samples. Confirms the new supervision wrapper does not disturb normal operation.
- **Test 2 — Genuine controlled internal error: not performed against the live HTTP server**, for a specific, honest reason: triggering a genuine panic requires either (a) modifying the real, working `run_workload`/Load/Stress/Spike/Volume logic (explicitly forbidden by this fix's own brief) or (b) a non-deterministic/production-unsafe production action. Neither is acceptable. The equivalent, real proof instead comes from `api_tests.rs`'s `phase6_error_a_genuine_internal_failure_is_detected_persisted_and_visible_live` (§17) — the exact same real, unmodified production code (`supervise_workload_task`, `finalize_performance_run_as_error`, `Store::finalize_performance_run`, the real HTTP router via `GET .../status`) exercised with a real, deterministic Tokio panic, just entered one level below a literal `curl` against the dev server. This is a deliberate, disclosed substitution, not a claim that live-HTTP verification happened when it did not.
- **Test 3 — Cancellation:** a real 20s Load test, cancelled via the real `POST .../cancel` endpoint after ~7s — `status: CANCELLED`, `verdict: CANCELLED`, `error_reason: null`. Confirms cancellation still resolves through the new supervisor's `Ok(())` branch, never misclassified as `ERROR`.
- **Test 4 — Restart recovery, via a genuine process crash:** started a real 20s Load run, then genuinely `Stop-Process -Force`'d the live orchestrator process mid-run (a real OS-level kill, not a simulated one) — confirmed via direct SQLite inspection that the row was left at `status = RUNNING` immediately after. Restarted the process; the real startup log showed `marked 1 performance test run(s) INTERRUPTED after a prior unclean shutdown`. Re-read via the real HTTP `GET .../status` endpoint: `status: INTERRUPTED`, `verdict: null` (not `"ERROR"`), a real recovery-authored `error_reason` describing why. Directly confirmed in the SQLite row too. This proves, live, that `ERROR` (task-level, detected while the process is up) and `INTERRUPTED` (process-level, detected only after a restart) are genuinely different real outcomes for genuinely different real failure classes — not the same code path relabeled.

20. Regression Results

#	Check	Result
1	`performance_testing.rs` tests	30/30 passed
2	Existing API execution tests (`api_tests.rs`, full file)	295/298 real passes, no-server run (3 pre-existing, unrelated, environment-dependent — §21); 298/298 with a live server
3	Existing cancellation tests	Passed — both the pre-existing `cancellation_actually_interrupts_a_running_workload_rather_than_waiting_it_out` unit test and the new `a_real_cancelled_workload_wrapped_in_supervision_still_reports_cancelled_not_error`
4	Existing persistence tests	`store_tests`, all passed, unaffected
5	Existing startup recovery tests	`engine_tests`' own `recover_incomplete_runs` tests, unaffected (different table); this fix's own new `list_incomplete_performance_runs`/`recover_incomplete_performance_runs` path re-verified live (§19 Test 4)
6	Full backend test suite	295/295 (no server) or 298/298 (live server), 0 regressions
7	Frontend relevant tests	`PerformanceTestingPage.test.tsx`, 7/7 passed
8	Full frontend suite	401/401 passed
9	Typecheck	Clean
10	Production frontend build	Succeeded, exit code 0
11	Backend build/check	`cargo check --lib`/`--tests`, clean

21. A Note on Pre-Existing, Environment-Dependent Test Failures

Unchanged from the original Phase 6 report's own disclosure (§32 there): `api_tests::phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`, `api_tests::phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`, `api_tests::phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request` depend on a real orchestrator instance already listening on `127.0.0.1:8080` (their own hardcoded `HEALTH_URL`). Re-confirmed here by running the identical full suite twice — once with no server up (these 3 fail with a real `CONNECTION_FAILED`), once with one up (all pass) — proving this fix introduced no new instance of this pattern and did not touch the affected tests, `HEALTH_URL`, or the `api-testing/execute` handler.

22. Known Limitations

- **Live, HTTP-triggered proof of the panic path was not obtained**, and is not obtainable, without either touching the untouched Load/Stress/Spike/Volume logic or taking a production-unsafe action — honestly substituted with an equally real, deterministic integration test (§19 Test 2, §17).
- **A full process kill is still, correctly, outside this fix's scope** — `supervise_workload_task` can only ever observe a task that is still running *within a live process*; if the whole process dies, the existing, unchanged `INTERRUPTED` recovery path is the only real detection mechanism, by design (§9, §19 Test 4).
- **The `is_cancelled()` (task-abort) branch of `SupervisionFailure::from_join_error`** is real, honest, and unit-tested-adjacent in spirit, but not reachable by any current real code path (nothing calls `.abort()` on the workload's `JoinHandle`) — included for completeness/defensiveness, not because it is presently exercised.
- **No raw panic backtrace or non-string payload is ever persisted** — a deliberate safety choice (§5, §11), not an oversight; internal `tracing::error!` logging (unchanged, existing convention) still carries the full message server-side for operator diagnosis.

23. Evidence Matrix

Capability	Code	Tests	Live Verified	Verdict
Normal Completion	`api.rs`/`performance_testing.rs`, unchanged	§17 (2 tests)	§19 Test 1	■
Threshold FAIL	`evaluate_verdict`, unchanged	pre-existing + §17 distinctness test	§19 (prior Phase 6 report)	■
Request Failure Distinction	`run_workload`, unchanged	pre-existing + regression	§19 (prior Phase 6 report)	■
Internal ERROR	`supervise_workload_task` + `finalize_performance_run_as_error`, new	§17 (5 tests)	§19 Test 2 (integration-level, disclosed)	■
Task Panic Handling	`SupervisionFailure::from_join_error`, new	§17 (3 tests)	§19 Test 2 (integration-level)	■
Partial Metrics	`finalize_performance_run_as_error`, new	§17.D	§19 Test 2 (integration-level)	■
ERROR Persistence	`Store::finalize_performance_run` (guarded), new guard	§17.D/§17.J/§17.H	§19 Test 2 (integration-level)	■
ERROR API Response	`PerformanceRunStatusResponse.error_reason`, new field	§17.D/§17.J	§19 Test 2 (integration-level)	■
ERROR Frontend Display	`#perf-execution-error` panel, new	§17 frontend test	Not live-browser-verified this fix (backend-focused fix; frontend contract additive-only, matches the already-live-browser-verified original page from the prior Phase 6 report)	■ (code + test)
Cancellation Regression	Unchanged	§17.G	§19 Test 3	■
Terminal Idempotency	`store.rs` guard, new	§17.H	Verified via direct DB inspection during Test 4	■
Startup Recovery INTERRUPTED	Unchanged	pre-existing	§19 Test 4 (real process kill)	■
No Orphan RUNNING	`supervise_workload_task`, new	§17.D	§19 Test 2 (integration-level) + Test 4 (recovery still catches a full-process-crash orphan)	■

24. Final Verdict

FIXED.

1. Genuine internal run failure reaches `ERROR` — yes, via real `JoinHandle` supervision (§8, §17.D).
2. Top-level task failure cannot silently leave a run `RUNNING` (within a live process) — yes; a full process kill is the one class this fix does not and cannot cover, and that class is correctly handled by the pre-existing, unchanged `INTERRUPTED` path instead (§19 Test 4).
3. `ERROR` is persisted — yes (§14).
4. `finished_at` is recorded — yes, frozen at real detection time (§13).
5. Safe `error_reason` is available — yes, sanitized, never a raw payload (§11).
6. Partial real metrics are preserved where available — yes, never invented, never discarded (§12).
7. `ERROR` is visible through API/frontend — yes (§15, §16).
8. `ERROR` is distinct from `FAILED`, `CANCELLED`, and `INTERRUPTED` — yes, proven both in code (§9) and live (§19 Tests 3 & 4).
9. Normal execution still works — yes (§17/§19 Test 1, zero regressions).
10. Cancellation still works — yes (§17.G/§19 Test 3).
11. Tests prove the lifecycle — yes, real, deterministic, non-flaky, no random/timing-based error injection anywhere (§17, §25 fake/stub audit).
12. No fake error state was introduced — confirmed by the fake/stub audit (§25): every `ERROR` this system can now produce comes from a real Tokio `JoinError`, detected by a real supervisor, for a real reason.

25. Fake/Stub Audit

Full keyword sweep (`fake|mock|demo|sample|placeholder|stub|dummy|simulated|hardcoded|Math\.random|setTimeout|TODO|FIXME|forced.success|forced.error|fake.error`, case-insensitive) against every changed file. Every hit is either a doc comment explicitly *negating* fake behavior (this codebase's established convention throughout every prior phase — e.g. "no `sleep`-simulated," "never fabricated," "not a mock"), a genuine HTML `placeholder` attribute, or unrelated pre-existing text (`store.rs`'s own `?` SQL-placeholder doc comment). Specifically confirmed:

1. `ERROR` is reached through genuine lifecycle handling — a real `tokio::spawn`, a real `JoinHandle`, a real `JoinError` from a real panic. No branch of `SupervisionFailure`/`finalize_performance_run_as_error` synthesizes an error without one having actually occurred.

2. No frontend-only fake `ERROR` state exists — `PerformanceRunStatusResponse.status`/`error_reason` are read directly from the real backend response; the frontend has no local state machine that can independently declare `ERROR`.

3. No random error injection exists in production code — `std::panic::panic_any`/`panic!` calls used to *test* the supervisor appear only inside `#[cfg(test)]` modules, never in `performance_testing.rs`'s or `api.rs`'s real, non-test code paths.

4. No timing-based flaky error test exists — every new test either awaits a real, deterministic future (a panic that always panics) or a real, bounded async operation (a genuine local HTTP target, a genuine cancellation with a generous timeout margin); none depends on a race window.

5. Partial metrics are not invented — `compute_metrics` is called with the exact, real `handle.samples` snapshot at detection time; §17.D asserts the exact real count (40), never a rounded/padded/estimated one.

6. `ERROR` is distinct from HTTP/request-level failure — `SampleOutcome::Error` (per-request) and `PerfRunState::Error`/`PerfVerdict::Error` (run-level) remain two entirely separate enums with no shared construction path; §17.C's regression test proves a run full of request-level failures still finalizes as a normal (non-`ERROR`) terminal state.

7. `ERROR` is distinct from threshold FAIL — `evaluate_verdict` (the only place `FAILED` is produced) is never called on the `ERROR` path at all (§9); `phase6_error_is_distinct_from_threshold_fail_and_from_cancelled` asserts the string values themselves differ.

8. `ERROR` is distinct from CANCELLED — proven both by code structure (mutually exclusive match arms) and live (§19 Test 3 vs the integration test's Test 2 equivalent).

9. `ERROR` is distinct from startup-recovery INTERRUPTED — proven live, §19 Test 4, with a genuine process kill.

10. Existing normal execution was not broken — §17/§20, zero regressions in 295-298 passing backend tests and 401 passing frontend tests.

26. Final Questions

1. **Why was `ERROR` previously unreachable or unreliable?** A detached `tokio::spawn` whose `JoinHandle` was never observed — if the workload task panicked, the finalization call sequenced after it never ran, and nothing else in the system would notice until the next restart's recovery sweep (§3, §7).

2. **Was the `JoinHandle` previously ignored?** Yes, confirmed by direct inspection of the pre-fix `tokio::spawn(async move { ... })` call, whose return value was discarded entirely (§3.F).

3. **What exact supervision mechanism was added?** `performance_testing::supervise_workload_task`, which spawns the workload as an inner task and awaits its real `JoinHandle`, converting `Err(JoinError)` into a real `SupervisionFailure` (§10).

4. **What happens when the workload returns an internal error?** There is no such return path by design — `run_workload` is infallible by construction; the only real internal-failure class this system has is a panic (§3.D, §7).

5. **What happens when the workload task panics?** The panic is caught by Tokio's own task isolation, surfaced as a real `JoinError`, converted to a safe message, and finalized as a real, persisted `ERROR` with real partial metrics (§8, §11, §17.E).

6. **Can a run still remain `RUNNING` after its supervised task exits?** No — the supervisor's `match` is exhaustive over `Ok`/`Err`; every exit of the inner task now reaches exactly one finalization call. (A full *process* death is a separate class, correctly handled by the pre-existing, unchanged `INTERRUPTED` recovery path instead — §22.)

7. **What happens to partial actual metrics?** Computed from the real, exact sample set collected before the failure, via the same unmodified `compute_metrics` every other path uses — never invented, never discarded (§12, §17.D).

8. **How is `error_reason` generated?** Extracted directly from the real Tokio panic payload when it is a plain string (the common case), or a fixed, safe generic fallback otherwise — never the raw, potentially-arbitrary payload (§11).

9. **Can `ERROR` become `COMPLETED` later?** No — enforced by a real DB-level guard (`WHERE status = 'RUNNING'`) in addition to the mutually-exclusive-by-construction call sites; proven directly (§13, §17.H).

10. **How is `ERROR` different from `FAILED`?** `FAILED` only ever comes from `evaluate_verdict` finding a violated threshold after a *normal* completion; `ERROR` is produced by the supervisor when the task itself never returned normally at all — `evaluate_verdict` is never even called on that path (§9).

11. **How is `ERROR` different from `CANCELLED`?** `CANCELLED` is a normal return from `run_workload` (the workers honored the stop signal and the function returned `()`); `ERROR` is the task *not* returning normally. Live-proven as two different real outcomes (§19 Tests 2 vs 3).

12. **How is `ERROR` different from `INTERRUPTED`?** `ERROR` is detected immediately, within the same live process, by a real task-level supervisor; `INTERRUPTED` is detected only after a restart, by a sweep with no live task to observe. Live-proven with a genuine process kill (§19 Test 4).

13. **Does startup recovery still work?** Yes, unchanged, re-verified live with a genuine process crash (§19 Test 4, §20 row 5).

14. **Was normal Load execution regression-tested?** Yes, both automated (§17 regression tests) and live (§19 Test 1).

15. **Was a genuine `ERROR` lifecycle test performed?** Yes — real HTTP round trip, real DB persistence, real reload, all via `api_tests.rs`'s new integration tests (§17, §19 Test 2).

16. **Was panic handling tested?** Yes — a real `panic!` and a real `std::panic::panic_any` inside real spawned tasks, both caught without crashing the process (§17.E).

17. **Were fake/simulated production errors introduced?** No — confirmed by the fake/stub audit (§25); every panic used in testing lives only inside `#[cfg(test)]` code.

18. **What limitation, if any, remains?** A genuine, full process crash (as opposed to an in-process task panic) is still, correctly, outside this fix's own detection mechanism and remains the domain of the pre-existing, unchanged startup-recovery-to-`INTERRUPTED` path — by design, since nothing running inside a dead process could ever supervise anything (§22).

End of report.